



Documento de Requisitos de Producto (PRD)

Prototipo de Sistema de Gestión de Inventario Autónomo y Seguridad Perimetral para Retail en Chinchá

1. Introducción y Contexto

Este proyecto tiene como objetivo traducir el sustento académico de la monografía "LRPD_InteligenciaArtificial.pdf" en un prototipo funcional, altamente interactivo y visual. Se modela un entorno de simulación bidimensional (2D) en forma de cuadrícula (grid) que representa el almacén de una empresa de retail en Chinchá Alta.

El sistema demuestra la convergencia de tres pilares de la Inteligencia Artificial moderna:

- 1. **Razonamiento Lógico:** Control de stock y reglas de seguridad perimetral a través de un motor declarativo en **SWI-Prolog v10.0.2**.
- 2. **Búsqueda Competitiva:** Toma de decisiones estratégicas ante un actor malicioso mediante el algoritmo **Minimax con Poda Alfa-Beta** con una profundidad de exploración de 4 niveles.
- 3. **Visión Artificial Profunda:** Detección en tiempo real de estantes vacíos e intrusos utilizando un pipeline de procesamiento de imágenes con **OpenCV** y clasificación profunda con **YOLOv8-nano**.

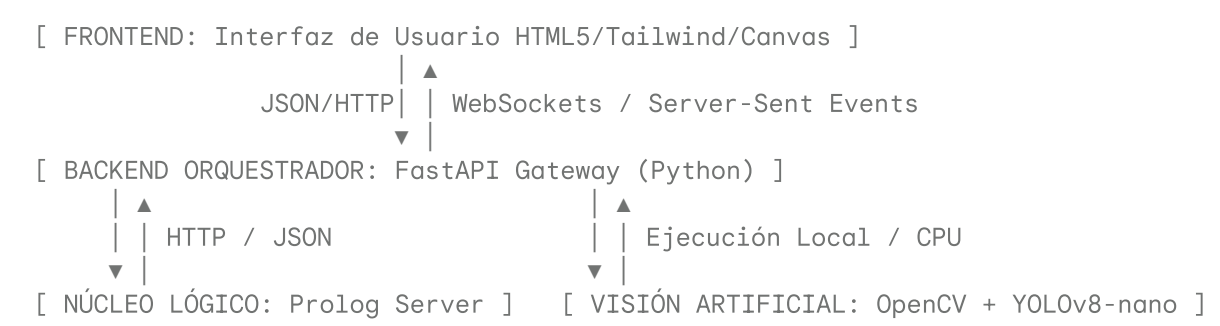
2. Estado Actual del Proyecto

Al momento de redactar este PRD, se ha completado con éxito la **Fase 1 (Inicialización)**:

- La estructura básica de directorios se ha creado de manera local.
- El repositorio de **Git** se ha inicializado localmente con su archivo `.gitignore` y sincronizado con el repositorio remoto de **GitHub**.
- El entorno virtual de Python (`venv`) se ha configurado de manera correcta.
- Se han instalado las dependencias de Python esenciales (`fastapi` , `uvicorn` , `requests` , `ultralytics` , `opencv-python-headless` , `pydantic`).

3. Arquitectura del Sistema

La solución se concibe como una aplicación web de escritorio local de alto impacto visual y bajo consumo de recursos, distribuida en tres capas principales:



3.1. Especificación del Mapa Grid (2D)

El almacén se representa matemáticamente como una matriz de dimensiones $M \times N$ (por defecto 5×5).

- **Valor 0 (Pasillo/Camino):** Área de libre tránsito para los agentes.
- **Valor 1 (Estante con Producto):** Zona de almacenamiento de mercancías.
- **Valor 10 (Estante Vacío / Quiebre de Stock):** Disparador crítico visual y lógico.
- **Valor 2 (Agente Guardián):** Entidad inteligente protectora.
- **Valor 3 (Agente Intruso):** Entidad inteligente adversarial.

4. Roadmap de Desarrollo (Fases de Construcción)

El agente de código asignado deberá implementar las siguientes fases de manera incremental, priorizando un enfoque de Producto Mínimo Viable (MVP):

Fase 2: El Cerebro Lógico (SWI-Prolog)

- **Objetivo:** Desarrollar el archivo de reglas `sistema_logico.pl` que exponga un servidor HTTP interno en el puerto `8080`.
- **Tareas:**
 1. Definir hechos dinámicos para productos, estados de estantes, cámaras y alarmas.
 2. Implementar reglas para la clasificación de amenazas:

$$\text{Amenaza} = f(\text{Clase Detectada}, \text{Zona comprometida}, \text{Horario})$$

3. Implementar el protocolo de reabastecimiento crítico.
4. Utilizar las primitivas `assert/1` y `retract/1` para modificar dinámicamente el conocimiento del almacén cuando el Gateway notifique actualizaciones.

Fase 3: El Motor Estratégico (Python & Minimax Alfa-Beta)

- **Objetivo:** Desarrollar el algoritmo de juego en un script `motor_estrategico.py` integrado con el Gateway FastAPI.
- **Tareas:**
 1. Representar el estado del juego como un vector estructurado.
 2. Diseñar la función de evaluación heurística (V) desde la perspectiva del Guardián:

$$V = w_1 \cdot \text{CoberturaValor} + w_2 \cdot (1/\text{DistanciaIntruso}) - w_3 \cdot \text{EstantesComp}$$

3. Implementar el árbol Minimax con Poda Alfa-Beta para calcular el mejor movimiento defensivo del Guardián, previendo las jugadas evasivas del Intruso a una profundidad de 4 pasos.

Fase 4: El Módulo de Visión (OpenCV + YOLOv8-nano)

- **Objetivo:** Simular e integrar el procesamiento de imágenes de seguridad en tiempo real.
- **Tareas:**
 1. Configurar un pipeline en Python que reciba la "imagen" (render virtual de una celda) y aplique escala de grises, filtro Gaussiano y binarización Canny.

2. Cargar el modelo YOLOv8-nano optimizado para clasificar las 4 clases principales descritas en "LRPD_InteligenciaArtificial.pdf".
3. Establecer un umbral ético estricto del 90% de confianza para decisiones automáticas directas. Si la confianza está entre 70% y 89%, derivar a confirmación manual del operario humano.

Fase 5: Frontend Interactivo y Copiloto Conversacional (Gemini API)

- **Objetivo:** Construir la interfaz de monitoreo visual en tiempo real e integrar un chat conversacional explicable.
- **Tareas:**
 1. Dibujar dinámicamente la cuadrícula utilizando HTML5 Canvas con suavizado de transiciones.
 2. Diseñar un panel moderno con Tailwind CSS que muestre el log de podas Alfa-Beta y alertas críticas.
 3. Conectar el backend con **Google AI Studio (Gemini API)** utilizando el SDK oficial gratuito. El chat recibirá del backend la cadena de inferencias lógicas de Prolog (ejemplo: `alerta_robo_activada -> estante_3_vacio -> sin_pago_registrado`) y la traducirá en un reporte comprensible para un operario humano.

5. Directrices sobre el Entorno Virtual (venv) para el Agente de Código

¡ALERTA DE DESARROLLO! El agente de código debe respetar rigurosamente el aislamiento de dependencias locales. Está terminantemente prohibido instalar software o ejecutar scripts de Python de manera global en el sistema operativo.

Cuándo USAR la activación del entorno virtual (venv):

- **Siempre** antes de instalar cualquier dependencia con `pip install`.
- Al ejecutar el servidor de desarrollo FastAPI (`python backend/main.py` o `uvicorn backend.main:app --reload`).
- Al ejecutar scripts de entrenamiento o inferencia de visión profunda con YOLOv8.
- Al realizar pruebas unitarias sobre los módulos de Python.

Cuándo NO usar el entorno virtual:

- Al ejecutar comandos nativos del sistema como `git` o controladores de base de datos nativos.
- Al inicializar e interactuar directamente con la consola interactiva de SWI-Prolog (`swipl`).
- Si el indicador de la terminal ya muestra el prefijo del entorno virtual activo, por ejemplo: `(venv) C:\ruta\proyecto>`. Volver a ejecutar el script de activación añade capas de procesos redundantes.

Ejemplos Prácticos de Comando Correctos vs. Incorrectos:

- **INSTALACIÓN DE LIBRERÍAS:**
 - ✗ *Incorrecto (Global):* `C:\Users\ADMIN> pip install fastapi`
 - ✓ *Correcto:*

```
C:\Users\ADMIN\retail-ia-chincha> .\venv\Scripts\activate  
(venv) C:\Users\ADMIN\retail-ia-chincha> pip install fastapi
```

- **EJECUCIÓN DEL BACKEND:**

-  *Incorrecto:* C:\Users\ADMIN\retail-ia-chincha> python backend/main.py
(Usará el intérprete global y generará error de "ModuleNotFoundError").
-  *Correcto:*

```
(venv) C:\Users\ADMIN\retail-ia-chincha> python backend/main.py
```

O de forma directa sin activar explícitamente el prompt:

```
C:\Users\ADMIN\retail-ia-chincha> .\venv\Scripts\python backend/main.py
```